

Module: ADS1 Algorithms and Data Structure 1
Instructors: Aicha KORICHI & Marwa MEISSA
Academic Year: 2025/2026
Class: ING1 Computer Engineering
Type: Tutorial

TD 04 Solutions

Exercise 1: Reverse Display

Write a C program that reads n integer values into an array and displays them in reverse order.

Listing 1: C Solution - Exercise 1

```
1 #include <stdio.h>
2
3 int main() {
4     // Variable declaration
5     int n;
6     // Use a maximum size for the array
7     int arr[100];
8
9     // Read the array size
10    printf("Enter the number of elements (max 100): ");
11    if (scanf("%d", &n) != 1 || n > 100 || n <= 0) {
12        printf("Invalid size.\n");
13        return 1;
14    }
15
16    // Read the array elements
17    printf("Enter the %d elements:\n", n);
18    for (int i = 0; i < n; i++) {
19        printf("Element %d: ", i + 1);
20        scanf("%d", &arr[i]);
21    }
22
23    // Display in reverse order
24    printf("\nDisplay in reverse order:\n");
25    for (int i = n - 1; i >= 0; i--) {
26        printf("%d ", arr[i]);
27    }
28    printf("\n");
29
30    return 0;
31 }
```

Exercise 2: Sum of Two Arrays

Write a C program to read two arrays of the same size and calculate a third array C where $C[i] = A[i] + B[i]$.

Listing 2: C Solution - Exercise 2

```
1 #include <stdio.h>
2
3 int main() {
```

```

4 // Variable declaration and initialization (for demonstration)
5 int n = 5;
6 int A[5] = {1, 2, 3, 4, 5};
7 int B[5] = {10, 20, 30, 40, 50};
8 int C[5];
9
10 // Calculate the sum C[i] = A[i] + B[i]
11 for (int i = 0; i < n; i++) {
12     C[i] = A[i] + B[i];
13 }
14
15 // Displaying the resultant array C
16 printf("Array A: {1, 2, 3, 4, 5}\n");
17 printf("Array B: {10, 20, 30, 40, 50}\n");
18 printf("Array C (Sum) :\n");
19 for (int i = 0; i < n; i++) {
20     printf("%d%s", C[i], (i == n - 1) ? "" : ", ");
21 }
22 printf("\n");
23
24 return 0;
25 }

```

Exercise 3: Check Sorting (Ascending)

Write a C program to check if a given array is sorted in ascending order.

Listing 3: C Solution - Exercise 3

```

1 #include <stdio.h>
2 #include <stdbool.h>
3
4 int main() {
5     // Declaration and initialization (Sorted example)
6     int n = 5;
7     int arr[5] = {1, 5, 12, 25, 30};
8     bool is_sorted = true;
9
10    // Traverse the array up to the second-to-last element
11    for (int i = 0; i < n - 1; i++) {
12        // If the current element is greater than the next, it is not sorted
13        if (arr[i] > arr[i + 1]) {
14            is_sorted = false;
15            break; // Exit the loop as soon as an unsorted pair is found
16        }
17    }
18
19    // Display the result
20    printf("Array checked: {1, 5, 12, 25, 30}\n");
21    if (is_sorted) {
22        printf("The array is sorted in ascending order.\n");
23    } else {
24        printf("The array is NOT sorted in ascending order.\n");
25    }
26 }

```

```
27     return 0;
28 }
```

Exercise 4: Merge Two Sorted Arrays (Descending)

Write a C program to merge two arrays of the same size, both sorted in descending order, into a new array sorted in descending order.

Listing 4: C Solution - Exercise 4/5

```
1  #include <stdio.h>
2
3  int main() {
4      // Declaration and initialization
5      int n = 5;
6      // Arrays sorted in descending order
7      int A[5] = {25, 20, 15, 10, 5};
8      int B[5] = {30, 18, 12, 8, 2};
9      int C[10]; // The merged array will have a size 2*n
10
11     int i = 0; // Pointer for Array A
12     int j = 0; // Pointer for Array B
13     int k = 0; // Pointer for Array C
14
15     // Merging the two arrays
16     while (i < n && j < n) {
17         // For descending order, we take the largest element
18         if (A[i] >= B[j]) {
19             C[k] = A[i];
20             i++;
21         } else {
22             C[k] = B[j];
23             j++;
24         }
25         k++;
26     }
27
28     // Copy remaining elements from A (if any)
29     while (i < n) {
30         C[k] = A[i];
31         i++;
32         k++;
33     }
34
35     // Copy remaining elements from B (if any)
36     while (j < n) {
37         C[k] = B[j];
38         j++;
39         k++;
40     }
41
42     // Displaying the merged array C
43     printf("Array A (Desc.): {25, 20, 15, 10, 5}\n");
44     printf("Array B (Desc.): {30, 18, 12, 8, 2}\n");
45     printf("Array C (Desc. Merge) :\n");
```

```

46     for (int l = 0; l < 2 * n; l++) {
47         printf("%d%s", C[l], (l == 2 * n - 1) ? "" : ", ");
48     }
49     printf("\n");
50
51     return 0;
52 }

```

Exercise 5: Rearrangement (Odds then Evens)

Write a C program that rearranges the elements of an array so that all odd numbers appear first, followed by all even numbers. The relative order does not need to be preserved.

Listing 5: C Solution - Exercise 6

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main() {
5      // Example test array
6      int T[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11};
7      int N = sizeof(T) / sizeof(T[0]);
8
9      // Pointers: L (Left), R (Right, which you called F)
10     int L = 0;
11     int R = N - 1;
12     int temp; // Variable for swapping
13
14     // --- Printing Original Array ---
15     printf("Original Array (N=%d): [", N);
16     for (int i = 0; i < N; i++) {
17         printf("%d%s", T[i], (i < N - 1) ? ", " : "");
18     }
19     printf("]\n\n");
20
21     printf("--- Partitioning Process ---\n");
22
23     // The core loop implementing your logic
24     while (L < R) {
25         // 1. Check L pointer: If T[L] is ODD, it's correctly placed, so advance L.
26         if (T[L] % 2 != 0) {
27             L++;
28         }
29         // 2. T[L] is EVEN (misplaced). Stop L and analyze R.
30         else {
31             // 2a. If T[R] is ODD (misplaced on the right), we have a swap pair.
32             if (T[R] % 2 != 0) {
33                 printf("Swapping %d (Even) with %d (Odd) at indices %d and %d\n",
34                     T[L], T[R], L, R);
35
36                 // Perform the swap
37                 temp = T[L];
38                 T[L] = T[R];
39                 T[R] = temp;
40

```

```

41         // Advance both pointers inward after the swap
42         L++;
43         R--;
44     }
45     // 2b. If T[R] is EVEN (correctly placed), advance R to find the required ODD swap partner.
46     else {
47         R--;
48     }
49 }
50 }
51
52 printf("--- Partitioning Complete ---\n");
53
54 // --- Printing Final Array ---
55 printf("\nFinal Array (Odd numbers first, Even numbers last):\n");
56 printf("[");
57 for (int i = 0; i < N; i++) {
58     printf("%d%s", T[i], (i < N - 1) ? ", " : "");
59 }
60 printf("]\n");
61
62 return 0;
63 }

```

Exercise 6: Find Pair with Sum S

Write a C program to find a pair of elements in an array whose sum is equal to a given value S .

Listing 6: C Solution - Exercise 7

```

1  #include <stdio.h>
2  #include <stdbool.h>
3
4  int main() {
5      // Declaration and initialization
6      int n = 6;
7      int arr[6] = {1, 4, 45, 6, 10, 8};
8      int S = 16;
9      bool found = false;
10
11     printf("Array: {1, 4, 45, 6, 10, 8}, Sum S = 16\n");
12
13     // Double loop to check every pair (Brute force  $O(n^2)$ )
14     for (int i = 0; i < n; i++) {
15         for (int j = i + 1; j < n; j++) {
16             // We ensure  $i < j$  to avoid duplicates and summing an element with itself
17             if (arr[i] + arr[j] == S) {
18                 printf("Pair found: (%d, %d) at indices (%d, %d)\n", arr[i], arr[j], i, j);
19                 found = true;
20                 // We could add 'goto end_loops;' here if we wanted to stop at the first pair
21             }
22         }
23     }
24
25     if (!found) {

```

```

26     printf("No pair found whose sum equals %d.\n", S);
27 }
28
29 return 0;
30 }

```

Exercise 7: Sum of Each Matrix Row

Write a C program to calculate and display the sum of each row of a matrix.

Listing 7: C Solution - Exercise 8

```

1  #include <stdio.h>
2
3  int main() {
4      // Declaration and initialization of the matrix (3x4)
5      int R = 3; // Number of rows
6      int C = 4; // Number of columns
7      int matrix[3][4] = {
8          {10, 2, 3, 4},
9          {5, 6, 7, 8},
10         {9, 10, 11, 12}
11     };
12
13     printf("Matrix (%dx%d) :\n", R, C);
14     for (int i = 0; i < R; i++) {
15         for (int j = 0; j < C; j++) {
16             printf("%4d", matrix[i][j]);
17         }
18         printf("\n");
19     }
20
21     // Calculation and display of the sum of each row
22     for (int i = 0; i < R; i++) {
23         int sum = 0;
24         for (int j = 0; j < C; j++) {
25             sum += matrix[i][j];
26         }
27         printf("Sum of Row %d : %d\n", i + 1, sum);
28     }
29
30     return 0;
31 }

```

Exercise 8: Maximum in a Matrix

Write a C program to find the maximum element in a matrix and display its value as well as its position (row and column).

Listing 8: C Solution - Exercise 9

```

1  #include <stdio.h>
2
3  int main() {
4      // Declaration and initialization of the matrix (3x4)

```

```

5  int R = 3; // Number of rows
6  int C = 4; // Number of columns
7  int matrix[3][4] = {
8      {10, 2, 90, 4},
9      {5, 6, 7, 8},
10     {9, 100, 11, 12} // 100 is the maximum
11 };
12
13 // Initialization with the first element
14 int max_val = matrix[0][0];
15 int max_row = 0;
16 int max_col = 0;
17
18 printf("Matrix (%dx%d) :\n", R, C);
19 for (int i = 0; i < R; i++) {
20     for (int j = 0; j < C; j++) {
21         printf("%4d", matrix[i][j]);
22     }
23     printf("\n");
24 }
25
26 // Traverse the matrix to find the maximum
27 for (int i = 0; i < R; i++) {
28     for (int j = 0; j < C; j++) {
29         if (matrix[i][j] > max_val) {
30             max_val = matrix[i][j];
31             max_row = i; // Record the row index
32             max_col = j; // Record the column index
33         }
34     }
35 }
36
37 // Display the result
38 printf("Maximum Element: %d\n", max_val);
39 // Displaying positions (1-based for user clarity)
40 printf("Position (Row, Column): (%d, %d)\n", max_row + 1, max_col + 1);
41
42 return 0;
43 }

```

Exercise 9: Matrix Transpose

Write a C program to compute the transpose of a 2D array (matrix).

Listing 9: C Solution - Exercise 10

```

1  #include <stdio.h>
2
3  int main() {
4      // Declaration and initialization of Matrix A (2x3)
5      int R_A = 2; // Rows of A
6      int C_A = 3; // Columns of A
7      int A[2][3] = {
8          {1, 2, 3},
9          {4, 5, 6}

```

```

10 };
11
12 // Matrix B (Transpose) will be (3x2)
13 int R_B = C_A; // Rows of B = Columns of A
14 int C_B = R_A; // Columns of B = Rows of A
15 int B[3][2];
16
17 printf("Original Matrix A (%dx%d) :\n", R_A, C_A);
18 for (int i = 0; i < R_A; i++) {
19     for (int j = 0; j < C_A; j++) {
20         printf("%4d", A[i][j]);
21     }
22     printf("\n");
23 }
24
25 // Transpose calculation: B[j][i] = A[i][j]
26 for (int i = 0; i < R_A; i++) {
27     for (int j = 0; j < C_A; j++) {
28         B[j][i] = A[i][j];
29     }
30 }
31
32 // Displaying the Transposed Matrix B
33 printf("\nTransposed Matrix B (%dx%d) :\n", R_B, C_B);
34 for (int i = 0; i < R_B; i++) {
35     for (int j = 0; j < C_B; j++) {
36         printf("%4d", B[i][j]);
37     }
38     printf("\n");
39 }
40
41 return 0;
42 }

```

Exercise 10: Matrix Multiplication

Write a C program to multiply two matrices (2D arrays).

Listing 10: C Solution - Exercise 11

```

1  #include <stdio.h>
2
3  int main() {
4      // Declaration and initialization of matrices
5      int R1 = 3, C1 = 2; // Matrix A (3x2)
6      int R2 = 2, C2 = 3; // Matrix B (2x3)
7
8      // Multiplication condition: C1 must equal R2 (2 = 2)
9      // The resultant matrix C will be (R1 x C2) = (3x3)
10
11     int A[3][2] = {
12         {1, 2},
13         {3, 4},
14         {5, 6}
15     };

```

```

16  int B[2][3] = {
17      {7, 8, 9},
18      {10, 11, 12}
19  };
20  int C[3][3]; // Result Matrix
21
22  printf("Multiplication of A (%dx%d) by B (%dx%d)\n", R1, C1, R2, C2);
23
24  // Check multiplication condition
25  if (C1 != R2) {
26      printf("Error: The number of columns in A must equal the number of rows in B.\n");
27      return 1;
28  }
29
30  // Matrix Multiplication: Triple Loop
31  for (int i = 0; i < R1; i++) { // Rows of the resultant matrix (R1)
32      for (int j = 0; j < C2; j++) { // Columns of the resultant matrix (C2)
33          C[i][j] = 0; // Initialize element C[i][j]
34          for (int k = 0; k < C1; k++) { // Common dimension (C1 or R2)
35              C[i][j] += A[i][k] * B[k][j];
36          }
37      }
38  }
39
40  // Displaying the Resultant Matrix C
41  printf("\nResultant Matrix C (%dx%d) :\n", R1, C2);
42  for (int i = 0; i < R1; i++) {
43      for (int j = 0; j < C2; j++) {
44          printf("%4d", C[i][j]);
45      }
46      printf("\n");
47  }
48
49  return 0;
50 }

```